

```

# FAQ Chatbot using NLP

import nltk
import string
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Download NLTK data
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab') # Added to resolve LookupError

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# -----
# FAQ Data
# -----

faq_data = {
    "What is your return policy?":
        "You can return products within 30 days.",

    "How can I track my order?":
        "You can track your order using the tracking link sent to your email.",

    "What payment methods do you accept?":
        "We accept credit cards, debit cards, UPI, and PayPal.",

    "How do I contact customer support?":
        "You can contact support at support@example.com.",

    "Do you provide home delivery?":
        "Yes, we provide home delivery across India."
}

questions = list(faq_data.keys())

# -----
# Text Preprocessing
# -----

stop_words = set(stopwords.words('english'))

def preprocess(text):
    # Convert to lowercase
    text = text.lower()

    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Tokenize
    tokens = word_tokenize(text)

    # Remove stopwords
    filtered_tokens = [word for word in tokens if word not in stop_words]

    return " ".join(filtered_tokens)

# Preprocess FAQ questions
processed_questions = [preprocess(q) for q in questions]

# -----
# Chatbot Function
# -----

def get_response(user_input):
    processed_input = preprocess(user_input)

    # TF-IDF Vectorization
    vectorizer = TfidfVectorizer()

    vectors = vectorizer.fit_transform(
        processed_questions + [processed_input]

```

```

)

# Calculate cosine similarity
similarity = cosine_similarity(
    vectors[-1],
    vectors[:-1]
)

# Best match index
best_match_index = similarity.argmax()

# Best similarity score
best_score = similarity[0, best_match_index]

# Threshold check
if best_score > 0.2:
    matched_question = questions[best_match_index]
    return faq_data[matched_question]
else:
    return "Sorry, I could not understand your question."

# -----
# Chat Loop
# -----

print("FAQ Chatbot")
print("Type 'exit' to stop.\n")

while True:

    user_question = input("You: ")

    if user_question.lower() == "exit":
        print("Chatbot: Goodbye!")
        break

    response = get_response(user_question)

    print("Chatbot:", response)

```

```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
FAQ Chatbot
Type 'exit' to stop.

You: payment options
Chatbot: We accept credit cards, debit cards, UPI, and PayPal.
You: exit
Chatbot: Goodbye!

```